

Diseño de Módulos y Componentes de Software

Actividad N°: 13 **Fecha:** 17/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

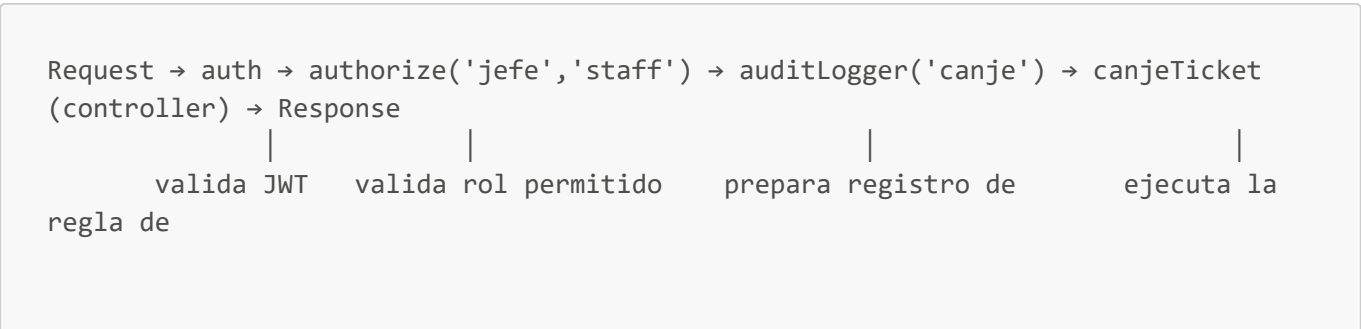
Definir la organización modular del código, tanto en el backend como en el frontend, especificando la responsabilidad de cada módulo y cómo se relacionan entre sí.

2. Módulos del Backend

Módulo	Carpeta	Responsabilidad
Configuración	src/config/	Conexión a MongoDB (<code>database.js</code>), logger (<code>logger.js</code>), resolución de la colección activa de tickets (<code>collections.js</code>)
Modelos	src/models/	Esquemas Mongoose: <code>Ticket</code> , <code>User</code> , <code>PuntoVenta</code> , <code>AuditLog</code>
Controladores	src/controllers/	Lógica de negocio: <code>ticketController</code> , <code>userController</code> , <code>puntoVentaController</code> , <code>auditController</code> , <code>authController</code>
Rutas	src/routes/	Definición de endpoints y composición de middlewares por ruta
Middleware	src/middleware/	<code>auth</code> (JWT), <code>auditLogger</code> (registro automático), <code>collectionSelector</code> (colección activa), <code>cache</code> (respuestas cacheadas)
Scripts	src/scripts/	Utilidades operativas: <code>setup.js</code> (importación completa), <code>importCSV.js</code> , <code>extractLocalidades.js</code> , <code>createAdmin.js</code> , <code>createIndexes.js</code>
Utilidades	src/utils/	Funciones auxiliares: <code>csvImporter.js</code> , <code>ecuadorTime.js</code> (manejo de zona horaria), <code>logger.js</code> , <code>auditLogger.js</code>
Entrada de la app	src/app.js	Configuración de Express, seguridad, CORS, Socket.IO y arranque del servidor

3. Cadena de middlewares por endpoint (diseño de composición)

Ejemplo real de la ruta de canje (`routes/tickets.js`):



responde	auditoría	negocio y
----------	-----------	-----------

Este diseño permite **reordenar o reutilizar** middlewares sin duplicar lógica: por ejemplo, `auth` y `authorize` se reutilizan en todas las rutas protegidas del sistema (tickets, users, audit, puntos-venta).

4. Módulos del Frontend

Módulo	Carpeta	Responsabilidad
Páginas	<code>src/pages/</code>	Una vista funcional por página: <code>Login</code> , <code>Dashboard</code> , <code>TicketsPage</code> , <code>UsersPage</code> , <code>AuditPage</code> , <code>PuntosVenta</code> , <code>ChangePassword</code>
Componentes	<code>src/components/</code>	<code>Layout</code> (envoltura general), <code>Navigation</code> (barra superior), <code>ProtectedRoute</code> (guard de rutas por rol), <code>RoleBasedRedirect</code> (redirección según rol tras login), <code>Unauthorized</code>
Contexto	<code>src/context/AuthContext.jsx</code>	Estado global de sesión: usuario autenticado, token, funciones de login/logout
Servicios	<code>src/services/</code>	<code>api.js</code> (cliente Axios con interceptores), <code>socket.js</code> (cliente Socket.IO como singleton), <code>index.js</code> (agregador)
Estilos	<code>src/styles/theme.css</code>	Paleta y estilos de marca (colores FTT)

5. Componente `ProtectedRoute` (diseño de control de acceso en frontend)

Actúa como una guarda declarativa alrededor de cada página sensible:

```
<Route path="/dashboard" element={
  <ProtectedRoute roles={['jefe']}>
    <Dashboard />
  </ProtectedRoute>
} />
```

Responsabilidad: verificar que exista sesión activa y que el rol del usuario esté incluido en `roles`; si no cumple, redirige a `/login` o `/unauthorized` según corresponda. Este control es complementario al `authorize()` del backend — **la validación real y definitiva ocurre siempre en el servidor**, el frontend solo mejora la experiencia de usuario ocultando opciones no permitidas.

6. Servicio `socket.js` (patrón Singleton)

Se diseña como una única instancia (`socketService`) compartida por toda la aplicación, con responsabilidad de:

- Establecer la conexión autenticada con token.
- Unirse a salas (`joinPuntoVenta`, `joinStaff`) según el rol.
- Registrar/limpiar listeners de eventos (`ticket-updated`) evitando fugas de memoria al desmontar componentes.

7. Matriz de trazabilidad módulo → requerimiento

Requerimiento	Módulo(s) responsables
RF-01 (login)	<code>authController</code> , <code>middleware/auth</code> , <code>AuthContext</code> , <code>Login.jsx</code>
RF-02/RF-03 (búsqueda/filtro)	<code>ticketController.getTickets</code> , <code>TicketsPage.jsx</code>
RF-04/RF-05 (canje individual)	<code>ticketController.canjeTicket</code> , modelo <code>Ticket</code>
RF-06 (canje masivo)	<code>ticketController.bulkCanjeTickets</code> (<code>bulkWrite</code>)
RF-07 (reimpresión)	<code>ticketController.reprintTicket</code>
RF-08 (auditoría)	<code>middleware/auditLogger</code> , <code>auditController</code> , modelo <code>AuditLog</code>
RF-09 (usuarios)	<code>UserController</code> , <code>UsersPage.jsx</code>
RF-10 (puntos de venta)	<code>puntoVentaController</code> , <code>PuntosVenta.jsx</code>
RF-11 (dashboard)	<code>ticketController.getTicketStats</code> , <code>Dashboard.jsx</code>
RF-12 (importación CSV)	<code>scripts/setup.js</code> , <code>scripts/importCSV.js</code> , <code>utils/csvImporter.js</code>
RF-13 (tiempo real)	Socket.IO en <code>app.js</code> , <code>services/socket.js</code>

8. Conclusiones del día

Se documentó la organización modular completa del sistema y su matriz de trazabilidad con los requerimientos, evidenciando que cada requerimiento aprobado tiene un módulo concreto responsable de implementarlo.

Observaciones: Sin observaciones.